

ARCHITECTURE AS A LIVING ASSET BASE

How we intend to produce, reuse and defend architecture work

Version 2 — a synthesis note for the architecture community

A knowledge base of typed assets, documents generated rather than written, and a system that asks experts what it needs instead of waiting to be told. Version 2 adds what building it taught us: where the questions come from, the mistake we made three times, and what it takes for a base to remain trustworthy once it starts producing its own material.

July 2026 — for discussion

Contents

1. The problem we are actually solving

Every engagement produces a substantial architecture document. Six months later, almost none of the thinking inside it is retrievable. The document is a PDF on a shared drive; the reasoning that produced it — why this vendor interface rather than direct access, why supervision rather than autonomy, what the effort really turned out to be — is either lost or lives in the head of whoever wrote it. The next engagement starts from a copy of the previous document and inherits its mistakes along with its structure.

Three failures compound. Knowledge decays because a document is a snapshot, not a state. Reuse is copy-paste, so a correction made once is not propagated. And proposals over-claim, because nothing distinguishes a capability we have delivered from one we hope to build — until a technical evaluation committee makes the distinction for us.

The response is not a better document template. It is to stop treating the document as the artefact.

2. The core idea

Assets are the source. Documents are renders. What we hold is a set of small, typed, versioned assets; a proposal or a design dossier is one projection of them at a point in time.

This is the same move we make routinely in engineering and rarely in architecture: separate state from presentation, keep the state structured, generate the presentation. A correction is then made once, in one asset, and every future document inherits it. A claim can be traced to the asset that supports it. And a machine can read the base precisely, because it reads whole typed objects rather than fragments of prose recovered by similarity.

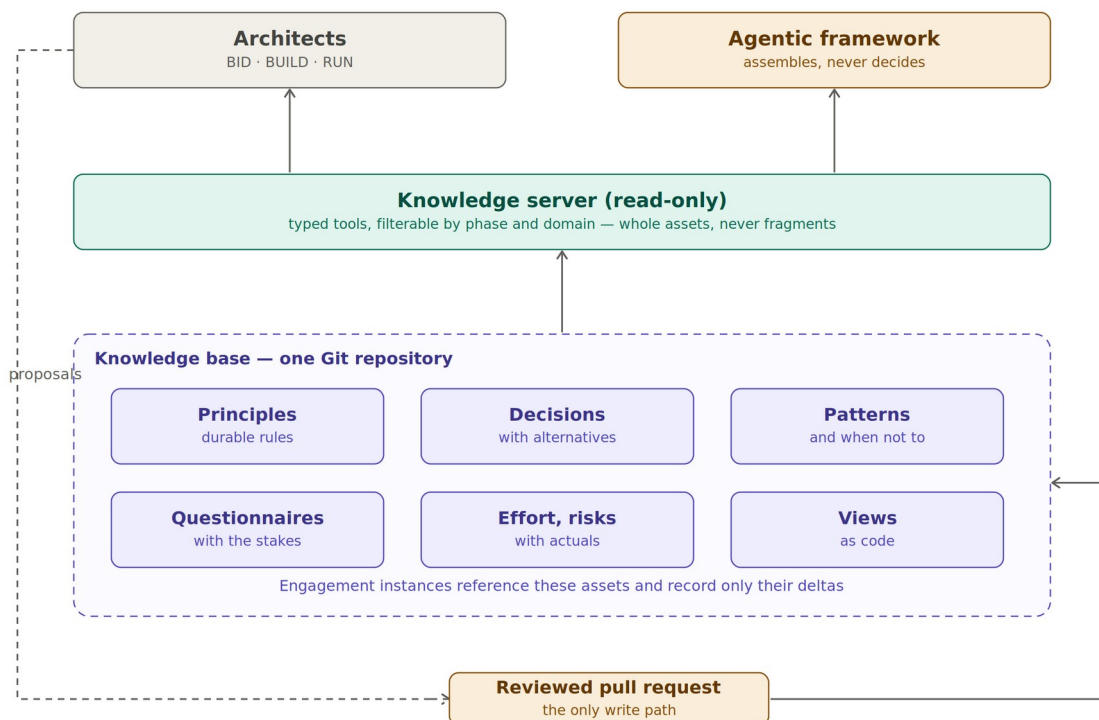


Figure 1 — The base, its read path and its single write path

Agents read through a server exposing typed tools, filtered by phase and domain, returning whole assets. They may draft, and they open a reviewed pull request to do so. They never write directly. That constraint is

what keeps the base a body of reviewed knowledge rather than a corpus that has quietly generated itself — a concern that turns out to run deeper than it first appears, and which section 10 returns to.

3. What the base holds

Asset type	What it captures	Why it survives the engagement
Principles	Rules that constrain future choices, each with a clause stating how to verify compliance	They outlive tools and vendors; they are the grid against which any design is reviewed
Decisions	A choice, the alternatives rejected, and the consequences accepted	Prevents re-litigating settled questions, and re-litigating them badly
Patterns	A repeatable solution, with an explicit section on when not to use it	The 'when not to' is what stops a pattern becoming dogma
Questionnaires	Due-diligence instruments where each question carries its stakes and the cost of a negative answer	Turns a one-off vendor audit into a reusable instrument
Effort models	Ranges with their variance drivers, and actuals recorded per engagement	The drivers matter more than the numbers; without actuals a model is a guess
Risk registers	Risks with mitigations and a record of where each actually materialised	Lets a bid team argue likelihood from evidence rather than instinct
Views	Diagram generators as code, parameterised	A new engagement changes labels; it does not redraw

Two mechanisms keep it small enough to be read. Nothing enters the reusable core from a single engagement: an asset is promoted only on its second occurrence, on a different engagement, stripped of anything client-identifying. And nothing is deleted: an asset that no longer holds is marked superseded and linked to its successor, because the trail of a reversed decision is often more instructive than the decision itself.

3.1 Confidence is data, not tone

Every asset declares whether what it asserts was directly observed, stated by a vendor, or assumed. This is the most consequential field in the schema, and it exists because of a specific failure mode: a language model restates an assumption with exactly the same assurance as a fact, and a reader cannot tell. Carrying the epistemic status as data means every tool response, every generated paragraph and every claim in a proposal can be qualified automatically.

Automation enforces it rather than trusting discipline. A vendor-stated assertion unreviewed for a year is downgraded to assumed by the validator. A draft asset surviving two harvests without promotion is deleted, because a permanently draft asset is noise.

At proposal time this becomes the claims register: one row per statement a customer could hold us to, each carrying an evidence level of proven, designed, vendor-stated or aspirational. Anything aspirational is

reworded as a trajectory the architecture supports, or dropped. Moderating a claim becomes a mechanical step in a review rather than an argument between an architect and a bid manager.

4. Where the questions come from

The single most useful clarification of the last few weeks, because without it the system looks like a black box that invents subjects. Three kinds of content are involved and they are easily conflated.

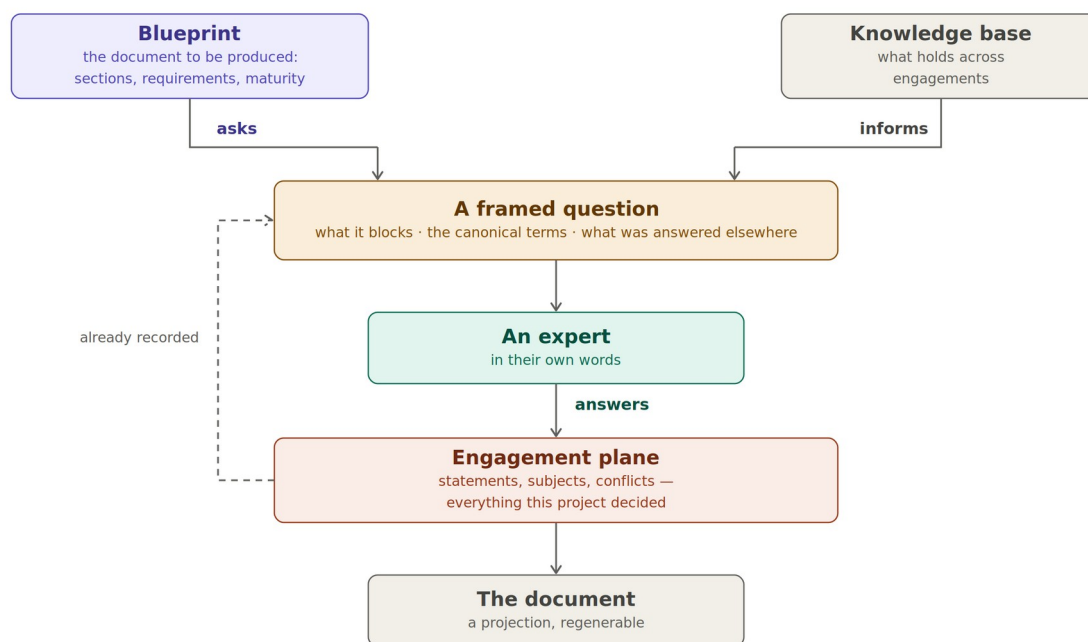


Figure 2 — The blueprint asks, the knowledge base informs, the engagement answers

The blueprint is the document to be produced: its sections, what each must answer, which subjects it requires and at which maturity. It is generic but of a different kind from knowledge — it is a shape to fill rather than something known. The knowledge base holds what is true across engagements. The engagement plane holds what this project has decided.

Gaps do not come from the knowledge base. They come from the blueprint: a gap is the distance between what a section requires and what the engagement holds. The knowledge base contributes the content of a question — the constraining principles, the prior answers, the candidate patterns — never its existence.

Stated that way it sounds obvious. It was not: the first implementation derived questions from a list written in the source, which is where section 9 begins.

5. How a document gets made

We built this in two iterations, and the second is where it landed.

The first was an orchestrated pipeline for a single author: an engagement brief plus the base, four passes — capability check, planning, drafting, assembly — with a human gate between each. It works, it is in service, and it remains the right tool when one architect owns a document end to end. Its most valuable output turned out not to be the prose but the gap list produced before drafting.

But real architecture work is collective, and applied to that situation the pipeline breaks twice: prose edited in parallel produces merge conflicts and, worse, semantic contradictions nobody notices; and a single author must interview everyone anyway, becoming a bottleneck who transcribes rather than an architect who decides. So the model inverts. The system holds the goal — fill in what is missing — and interrogates the experts. A chatbot in reverse: the initiative sits with the process, the knowledge with the humans.

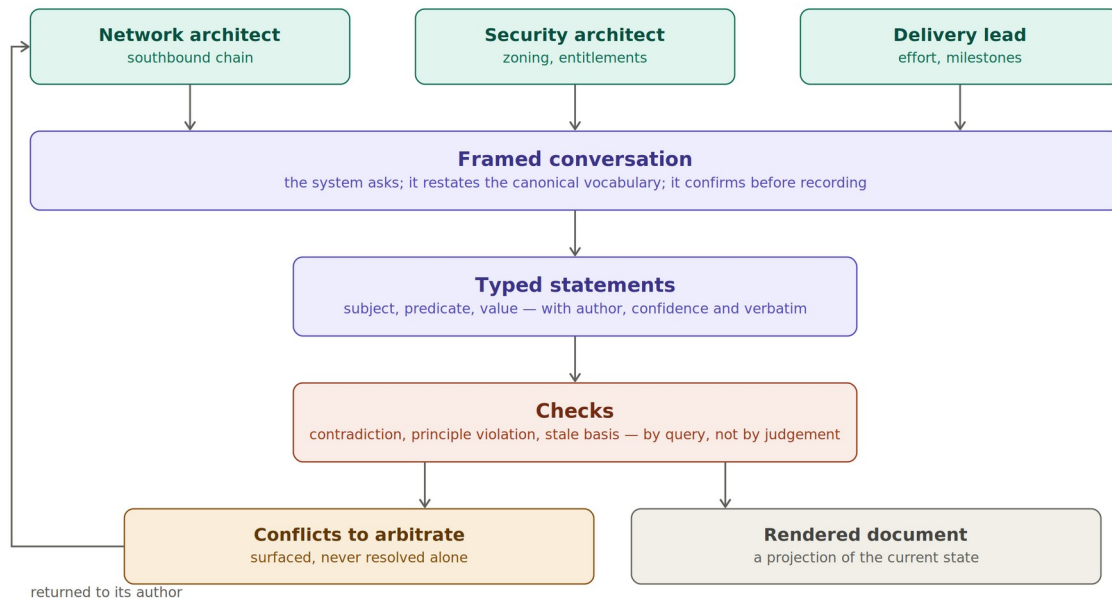


Figure 3 — Multi-expert contribution: framed conversation, typed statements, checks, arbitration

5.1 Statements, not prose

The unit of contribution is not a paragraph. Each contribution is recorded as typed statements — a subject, a predicate from a controlled list, a value — plus the author, the confidence and the expert's own words kept verbatim. Prose is generated from the statements; contradictions become detectable by query rather than by proofreading.

5.2 The question is where the quality is

Crystallising a gap means producing something answerable in one or two sentences by one identified person, carrying four things: why it matters and what it blocks, the canonical vocabulary to use, the shape of the expected answer, and — where the base has one — what a previous engagement answered. That last element is the reuse mechanism: the expert confirms or diverges, and the delta becomes the contribution.

Presenting the canonical vocabulary also solves the hardest problem in the design. If one expert says 'the management cluster' and another 'the administration zone', no contradiction is ever detected because the system does not know they mean the same thing. Serving the term with the question makes the answer attach to the right subject by construction — elicitation is the normalisation mechanism.

5.3 Two ways a disagreement surfaces

A contributor can contest a statement explicitly: they read a draft, disagree, and say so. That is a declaration, and it surfaces the disagreements people already know they have. The harder and more valuable case is the one nobody notices — two experts answering different questions, weeks apart, recording statements that cannot both hold. Those are found by query, not by anyone reading carefully. Both are recorded with their origin, so that the second capability can be tested rather than assumed.

6. Refinement, and seeing what is left

A first design gave the whole engagement one cadence: structural questions, then mechanisms, then parameters. It was wrong, and usefully so. Subjects do not mature at the same rate: one may be at parameter level while another has barely been named. Refinement happens per subject, each at its own pace.

Level	The question asks	It advances when	The base contributes
L0 named	nothing — the term exists, no more	a statement mentions it	the canonical term
L1 framed	what it is for, its boundary, what must hold	the problem is stated, not the solution	the principles that constrain it
L2 decomposed	what parts it breaks into	each part exists as a subject	candidate patterns
L3 decided	which mechanism for each part	one decision per part, alternatives recorded	prior decisions and their consequences
L4 specified	which values and thresholds	parameters are recorded	values from previous engagements

The level that carries the idea is L2. When an expert has just decomposed a subject, the base can say: this resembles a pattern we hold, here is how it was broken down elsewhere, and here is the section that says when not to use it. Problem and solution refine together, which is what happens in a good design review and almost never in a document.

Two consequences we did not anticipate. A decomposition is generative: it creates the subjects the next questions talk about, so refinement produces its own next steps. And refinement is not monotonic — an arbitration can invalidate a framing, in which case the subject is demoted, statements above the new level are flagged rather than deleted, and the reopened questions carry the previous answers as context. A demotion must not feel like a punishment for having answered.

Subject	L0 named	L1 framed	L2 decomposed	L3 decided	L4 specified	Blocked by	Sections
management-cluster					now	—	5.2 · 6.3
fallback-plan				now		Q-0014 · bob	8.5 · 10.4
observation-plane				now		—	8.1 · 9.1
mcx-interworking		now				Q-0021 · alice △ 9 d	4.2 · 4.3
break-glass			now			Q-0018 · charlie	6.3
sim-lifecycle	now					not started	4.4

How to read this board

Filled cells are levels reached; the amber cell is where the subject stands now. A section renders as final only when its subjects are decided or beyond; below that it renders as provisional. The column that matters in a weekly review is not the levels but 'blocked by': a subject stalled at the same level for days, and the name of whoever owes an answer.

Figure 4 — The maturity board: what is ripe, what is stuck, and with whom

Two derived rules make the board operational. A section renders as final only when every subject it depends on is decided or beyond; below that it renders provisional and names what is not ripe. And a subject

unchanged beyond a threshold with an open question is flagged. In a weekly review the column that matters is not the levels — it is 'blocked by', with a name and an elapsed time.

7. A worked example

Three architects, one contested decision, no command line. The system opens an issue and assigns it to a role; this is what Alice sees.

[Q-0001] role:cloud-architect · section 5.2 · blocks 2

Which storage backs the management cluster, and is it distinct from the storage of the workload clusters?

Why this matters

P-009 requires an independent failure domain. Without a distinct backing store, section 6.3 cannot claim that the recovery chain survives an incident affecting the workloads it is meant to recover.

Please use these terms

management-cluster · workload-cluster · failure-domain

Previously answered elsewhere

mcx-core-2026 → “dedicated array, not shared with workloads” — confidence: designed

Confirm this for your engagement, or tell us how it differs here.

Context what is already recorded about management-cluster · the current draft

Alice diverges: the customer already operates a tier-one array and wants a single storage contract. The system records nothing yet — it shows her what it intends to write, with two statements carrying different confidence, and her own words preserved beneath. She confirms. A check produces an advisory note: a shared array sits uneasily with the independent failure domain the principle requires. The system neither blocks nor decides.

Three days later Bob, who was never asked this question, contests a statement about a subject he owns. Both positions stay active, a conflict opens, and Charlie arbitrates with a mandatory rationale: the principle is not negotiable here, and the commercial expectation goes into the risk register rather than into the design. Bob's position is kept, Alice's is amended and its previous wording stays in the history.

The rejected option appears in the final document because arbitration recorded it, not because somebody remembered to write it down. That is the difference between a document and a record.

8. Under the hood

What the expert sees is a question and, later, a proposal of what will be recorded. Everything else is the system's problem.

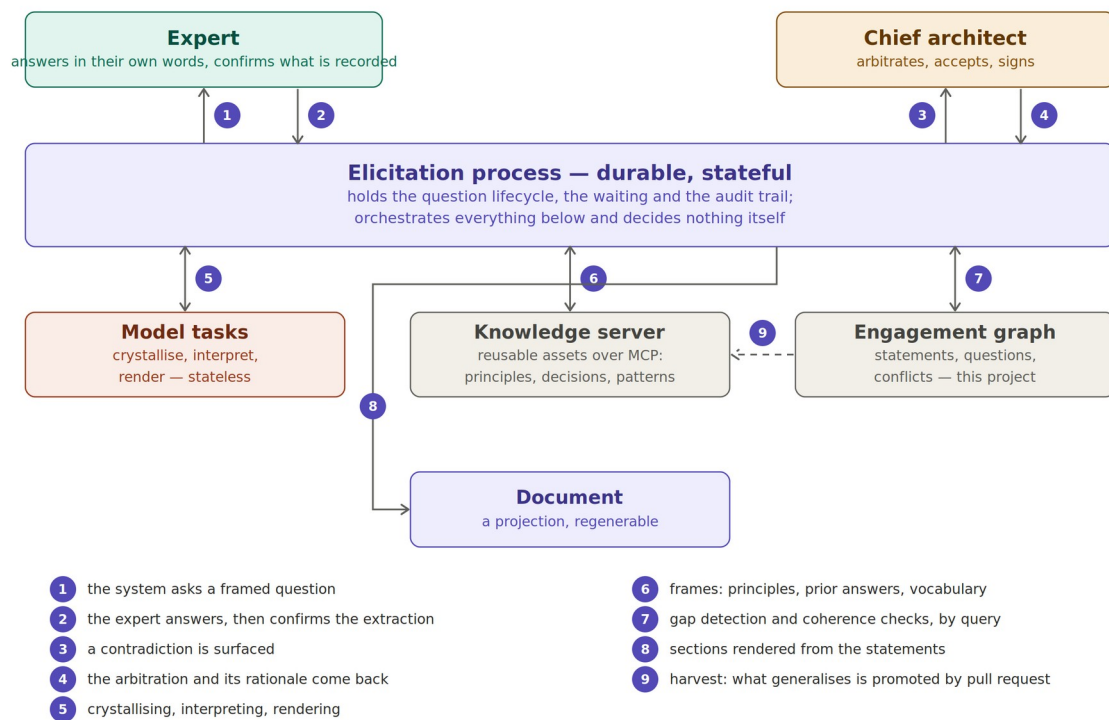


Figure 5 — One exchange, end to end: who asks, who reasons, who holds the truth

Two separations make this dependable. The first is between what holds state and what reasons. A conversation unfolds over days: a question goes out on Tuesday, one expert answers Thursday, another is on leave. A process that must stay alive through that is broken at the first absence. So the durable process owns the state, the waiting and the audit trail; agent tasks are short, stateless and replaceable; and humans are external actors reached through a mailbox, not functions to be called and awaited.

8.1 The determinism boundary

The second separation decides whether the whole thing is trustworthy.

Concern	How it is done	Why
Detecting a gap	Query	A model asked to find gaps invents some and, worse, misses others silently
Detecting a contradiction	Query	Same, and the result must be reproducible
Advancing a maturity level	Repository, after human confirmation	Progress is a fact about the engagement, not an impression
Phrasing a question	Model	This is writing, which is what it is good at
Proposing statements from an answer	Model, then human confirmation	Extraction is a proposal, never a commit
Proposing a pattern at decomposition	Model retrieves, human accepts or discards	A pattern applied automatically is dogma with a database behind it
Residual semantic coherence	Model, advisory only	Useful, imperfect, therefore never a gate

Concern	How it is done	Why
Resolving a conflict	Human only	Accountability cannot be delegated to a probabilistic component

One rule follows and admits no exception: a model never writes to the base. It proposes; a human commits, and the commitment is recorded with a name against it.

9. Derived, not asserted

The most useful thing building this taught us, and we learned it three times in three different disguises.

The gap detector, on inspection, held a list of thirty-five specifications naming every question the system could ever ask. It was not detecting anything; it was enumerating a fixture, and the twenty-six 'held gaps' its reports proudly counted were twenty-six rows of that table. The acceptance suite written to verify it then turned out to contain twenty-two literal section identifiers, hard-coding the very thing it was meant to check. And the blueprint, built from memory of a template rather than checked against it, reused the template's numbering while assigning fourteen identifiers to different subjects — it looked like the document and said something else.

Each time, something that should have been derived had been asserted. And each time the fix was the same: make it data, and make the derivation testable.

Which gives a test that separates a knowledge system from a clever fixture, and it is embarrassingly simple: ****change the input and see whether the output changes.**** Add a section to the blueprint and a gap must appear; remove it and the gap must go. Rename a subject and nothing must break. If the output is stable under changes to the input, whatever is producing it is not deriving it.

It is worth naming because the failure is not one of competence. Hard-coding is what you do to make something work today, and it survives because it produces plausible output. A system that enumerates a fixture and a system that computes from a model are indistinguishable from their reports — until the day you need to change something, which is the day you needed the system for.

10. Integrity: provenance and circular evidence

A base that generates documents can be fed its own documents, and that innocuous-sounding capability contains the failure mode most likely to destroy the whole endeavour.

A statement asserts something. A generated document repeats it, because it was rendered from that statement. Ingest the document and a second statement appears, asserting the same thing and citing the document. The base now appears to hold two independent sources for one claim, and its confidence looks stronger than it is. Nothing in the natural implementation would notice — parse, propose, record is exactly how one would build it.

This is the mechanism by which an evidence-based knowledge base becomes a self-referential one. It costs nothing to prevent and is very hard to detect afterwards.

The defence has three layers, and only the first was already in place. Confidence as data says what each assertion rests on. Provenance says where a document came from: every generated document carries a block naming the blueprint, the engagement, the base commit and the statements it was rendered from, so

a returning document is recognisable. And independence — the layer we had not thought about — says that two sources corroborate only if neither derives from the other. A second source repeating what the first said adds a citation to the existing statement, never a second statement.

Feeding a document back then becomes something useful rather than dangerous. A derived document produces no statements at all; it produces a drift report saying which passages no longer match the statements they came from, and — the valuable part — which passages are in the document and in nothing else, which is how a hand edit to a generated Word file comes back to the surface.

The concern generalises beyond documents. As more of what the base contains is produced with the help of the base, the distinction between what we know and what we have restated erodes quietly. Provenance and independence are what keep that distinction alive, and they only work if they are in place before the volume makes them necessary.

11. Measuring the thing itself

Until recently every judgement about quality here was a matter of opinion: is this question well framed, is this extraction faithful, is this pattern well chosen. Feeding a document back gave us the first metric that is not.

Take the document from which the base was originally written — in our case a twenty-two page architecture dossier of about seven thousand words — and feed it in. Measure how many of the assets it produced are recovered by crystallisation. That recovery rate is a quality metric for the pipeline, and it moves when the prompts move. A change that raises recovery from sixty-eight per cent to eighty is progress; one that reads better and lowers it is not.

The assets that are not recovered are the more instructive output. Most fail for one reason: they were stated across three paragraphs rather than in one place, which is what prose does and precisely what structured assets exist to prevent. The benchmark therefore measures two things at once — how well the pipeline reads, and how much a document loses when knowledge is dissolved into narrative.

Other measurements become available once the apparatus exists: the false-positive rate of detection rules, the proportion of questions answered without a follow-up, the time between a question being sent and answered per role. None of them replaces judgement. All of them replace argument about facts that can be counted.

12. How the base grows

The honest answer to what you get out of this depends on whether you are first or third. Going first is expensive: everything is written from scratch, nothing is offered as a default, and the effort model is a guess because it has no actuals behind it.

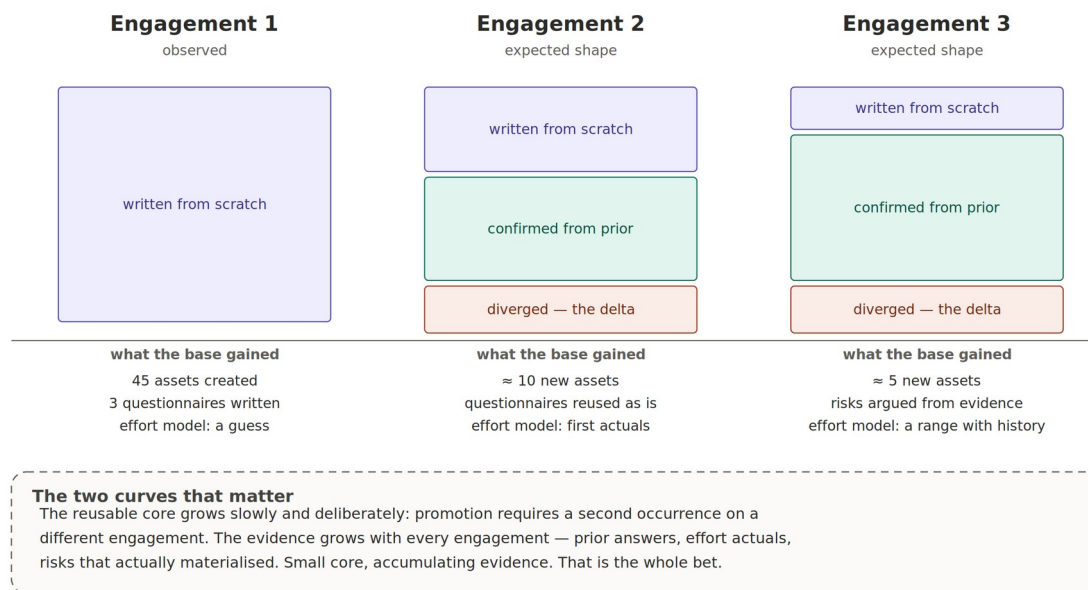


Figure 6 — The composition of an engagement's work, and what the base gains from it

From the second engagement the shape changes. Questions arrive pre-filled with what was answered before, so only the divergence has to be thought about. The effort model receives its first actuals, which is the precise moment it stops being an opinion. By the third, risks can be argued from evidence rather than instinct.

The reusable core grows slowly and on purpose — promotion requires a second occurrence on a different engagement. The evidence grows with every engagement. A small core with accumulating evidence is the whole bet; a large core nobody reads is the failure mode.

One caveat, in keeping with what this note preaches about confidence. The first column is observed. The second and third are an expected shape, not a measurement, and will remain a hypothesis until a second engagement tests it.

13. The working principles

Distilled, this is what we are committing to. Three were added by building it.

- **Assets are the source, documents are renders** — what we hold are typed assets; a proposal or a dossier is one projection of them.
- **Confidence is data, not tone** — verified, vendor-stated or assumed travels with every assertion, and automation enforces its decay.
- **The blueprint asks, the base informs, the engagement answers** — a gap is the distance between what a section requires and what the engagement holds. The knowledge base informs a question; it never creates one.
- **Nothing enters the core from one engagement** — promotion requires a second occurrence on a different engagement; the core stays small enough to be read.
- **Subjects are refined coarse to fine** — a new subject is framed, then decomposed, then decided, then specified, each at its own pace, and it may go backwards.
- **The system surfaces, humans arbitrate** — conflicts, gaps and inconsistencies are raised, never resolved automatically.

- **The model proposes, a human commits** — extraction, drafting, detection and pattern retrieval are proposals; commitment is a human act, recorded.
- **Determinism where it matters, generation where it helps** — correctness by query, writing by model. Never the reverse.
- **Derived, not asserted** — change the input and the output must change; otherwise what looks like a knowledge system is a fixture.
- **Independence before corroboration** — two sources corroborate only if neither derives from the other. A repetition adds a citation, not a second statement.
- **Harvest, or it is an archive** — effort actuals and materialised risks are the two things no amount of architecture thinking can produce.

14. Roles and state of play

Role	What it owns
Contributor	Answers questions in their domain, in their own words; confirms or corrects the extraction. May decline with a reason, or reroute — a declared unknown is information
External contributor	Offers material without being asked. Confirms the meaning of what is extracted from it; never writes to the graph
Section owner	Completeness and coherence of a section; triages contributions; escalates what cannot be resolved between contributors
Chief architect	Arbitrates conflicts, accepts statements, decides deferrals, owns the claims register and therefore the level of promise, signs the harvest
Core owner	Reviews changes to the reusable core in one domain; arbitrates promotions; re-verifies vendor-stated assertions
Maintainer	Repository hygiene, schema, automation, releases

One caution about the chief architect, because the drift is predictable: they must not answer the questions on the experts' behalf. They know the subject and they are faster, and doing so quietly converts a collective instrument back into a single-author document with extra steps. The tooling records it when they do.

14.1 What exists today

Component	State	What you can do with it
Knowledge base	In service	Typed assets validated automatically on every change
Knowledge server	In service	Read tools over the base, filterable, returning whole assets with their confidence
Authoring pipeline	In service	Produce a document from the base plus a brief, in four passes with a human gate between each
Blueprint	In service	The real document structure, 49 sections, titles verbatim from the bid template
Elicitation prototype	Implemented, being corrected	Gap detection, framed questions, confirmation before recording, declared and detected conflicts, arbitration,

Component	State	What you can do with it
		assembly
Acceptance suite	In place	Nineteen tests producing a progression report generated from the graph, not written by hand
Document ingestion	Specified	Drift reports on returning documents, and the recovery benchmark
Mailbox rendering	Specified	The same cards in issues or by email, so contributors need install nothing
Web interface	Not started	Deliberately. Three screens are sketched; they will be built when usage justifies them

14.2 How we test our own claims

Three episodes, reported because they are the discipline rather than the result. The reference scenario was written as a narrative before anything ran, and writing it exposed a defect no reading of the specification had — the contradiction rule would not have detected the very conflict the story turns on. The scenario file now carries a banner saying it is fabricated and a section listing where the real run was expected to diverge, written before the run so it could not be adjusted afterwards. The acceptance suite's most important assertion is that the reference engagement finishes marked provisional: a system reporting a document complete while four subjects are unframed is lying to its user, and a test that accepts it is complicit. And the blueprint was rebuilt after being compared to its source document rather than remembered.

15. Where this leads

Reading back over the last two months of design, the thing being built is not a documentation tool, and calling it one undersells both the value and the difficulty.

What it is, is an institutional memory with an integrity model. Every claim carries provenance and an epistemic status. Derivation is tracked, so corroboration can be distinguished from restatement. Disagreement is a first-class object, arbitrated in writing rather than settled in a meeting nobody minuted. The system knows what it does not know and goes to find out. And its own quality is measurable, which means it can be improved on evidence rather than on taste.

Those are the properties of a scientific apparatus rather than of a knowledge management system, and that is the interesting part. They are also, taken seriously, uncomfortable — which brings the honest question.

This apparatus makes visible a number of things that ambiguity currently protects: who decided what and when, what we do not know, what we promised without evidence, and which expert has owed an answer for nine days.

Every one of those is an improvement in the abstract and a friction in the concrete. An architect who has always resolved disagreements verbally will find arbitration-with-a-written-reason slow. A bid team will find the claims register inconvenient at precisely the moment it matters most. And a board showing that a subject has been stuck for nine days with a named owner is a management instrument, which is a different thing from an architecture instrument and will be read as one.

So the real adoption risk is not technical and never was. It is whether an organisation wants this much visibility into its own reasoning. The answer will differ by team and by engagement, and the honest position

is that the apparatus should earn its place one property at a time: the gap list first, because nobody objects to being told what they do not know; then the claims register, because losing a bid for over-claiming is a shared enemy; and only then the parts that record who said what.

Where it leads, if it works, is a shift in what gets audited. Today an audit reads the document. With this, the document is a projection and the base is the thing to audit — which is both more honest and considerably more exposing. That is the destination, and it is worth being clear-eyed that it is a destination some will not want to reach.

16. Try it

The cheapest way to find out whether this is useful to you costs about half a day and commits you to nothing.

- **Read the base** — clone the repository and run the capability check. Five minutes.
- **Describe your engagement** — fill the engagement brief for something you are working on now. This is the part people skip, and a thin brief produces a generic document no amount of prompting will rescue. Two hours, and useful even if you stop there.
- **Ask it what is missing** — run the planning pass and read the gap list: what the base cannot cover, what your brief is missing, which vendor answers you are quietly assuming. You get that before drafting a line.
- **Then decide** — if the gap list was useful, draft a section. If it was not, tell us why — that is a more valuable contribution than a section.

For an engagement with several contributors, the elicitation prototype runs the three-architect scenario locally: one machine, several roles, nothing to install for anyone else. It is the fastest way to see whether the model matches how your team actually argues.

The assets currently carry the reasoning of a single engagement. They become shared knowledge on the second — which is to say, when one of you tries this on something we have never seen.